

ASSIGNMENT 7.5

Hemavathi N

2303A51965

Batch 24

AIAC

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

C: > Users > ADMIN > OneDrive > Desktop > ce

```
1 def add_item(item, items=[]):
2     items.append(item)
3     return items
4     print(add_item(1))
5     print(add_item(2))
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > 
1 #dont use mutable default arguments
2 def add_item(item, items=None):
3     if items is None:
4         items = []
5     items.append(item)
6     return items
7 print(add_item(1,[78,90]))
8 print(add_item(2,[90,23,34]))
```

OUTPUT:-

```
PS C:\Users\ADMIN> & "C:/P
[78, 90, 1]
[90, 23, 34, 2]
PS C:\Users\ADMIN>
```

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():
```

```
    return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Expected Output: Corrected function

```
C: > U: C:\Users\ADMIN\OneDrive\Desktop\certificates\LAB
1 def check_sum():
2     return (0.1 + 0.2) == 0.3
3     print(check_sum())
4
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.py > ...
1 def check_sum():
2     return abs(0.1 + 0.2 - 0.3) < 1e-10
3     print(check_sum())
```

OUTPUT:-

```
PS C:\Users\ADMIN> & "C:/Prog
True
PS C:\Users\ADMIN>
```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
```

```
    print(n)
```

```
    return countdown(n-1)
```

```
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

```
C: > Users > ADMIN > OneDrive > Desktop > cer
1 def countdown(n):
2     print(n)
3     return countdown(n-1)
4     countdown(5)
5
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.py > cou
1 #Correct recursion with stopping condition.
2 def countdown(n):
3     if n <= 0:
4         print("Blastoff!")
5     else:
6         print(n)
7         countdown(n-1)
8     countdown(5)
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB
print(n)
^^^^
IndentationError: expected an indented block after functi
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
5
4
3
2
1
Blastoff!
PS C:\Users\ADMIN>
```

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():
data = {"a": 1, "b": 2}
return data["c"]
print(get_value())
```

Expected Output: Corrected with .get() or error handling.

```
C:\Users\ADMIN> OneDrive\Desktop\certificates\LAB
1 def get_value():
2 data = {"a": 1, "b": 2}
3 return data["c"]
4 print(get_value())
```

Corrected code:

```
C:\Users\ADMIN> OneDrive\Desktop\certificates\LAB
1 #Corrected with .get() or error handling
2 def get_value():
3 data = {"a": 1, "b": 2}
4 return data.get("c", "Key not found")
5 print(get_value())
```

OUTPUT:

```
IndentationError: expected an indented block after functi
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
File "c:\Users\ADMIN\OneDrive\Desktop\certificates\
data = {"a": 1, "b": 2}
^^^^
IndentationError: expected an indented block after fu
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
Key not found
PS C:\Users\ADMIN>
```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop
def loop_example():

i = 0

while i < 5:

print(i)

Expected Output: Corrected loop increments i.

```
C: > Users > ADMIN > OneDrive > Desktop
1  def loop_example():
2  i = 0
3  while i < 5:
4  print(i)
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5
1  #correct the loop incerments i
2  def loop_example():
3      i = 0
4      while i < 5:
5          print(i)
6          i += 1
7  loop_example()
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users\ADMIN/OneDrive/Desktop/certificates/LAB 7.5.py", li
i = 0
^
IndentationError: expected an indented block after function definiti
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users\ADMIN/OneDrive/Desktop/certificates/LAB 7.5.py", li
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users\ADMIN/OneDrive/Desktop/certificates/LAB 7.5.py", li
loop_example()
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users\ADMIN/OneDrive/Desktop/certificates/LAB 7.5.py", li
0
1
2
3
4
PS C:\Users\ADMIN>
```

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

```
C: > Users > ADMIN > OneDrive > Desktop
1  a, b = (1, 2, 3)
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.py > ...  
1  #Correct unpacking or using _ for extra values.  
2  a, b, _ = (1, 2, 3)  
3  print(a)  
4  print(b)
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python  
Traceback (most recent call last):  
  File "c:\Users\ADMIN\OneDrive\Desktop\certi  
    a, b = (1, 2, 3)  
    ^^^^^  
ValueError: too many values to unpack (expect  
PS C:\Users\ADMIN> & "C:/Program Files/Python  
PS C:\Users\ADMIN> & "C:/Program Files/Python  
1  
2  
3  
PS C:\Users\ADMIN> & "C:/Program Files/Python  
1  
2  
PS C:\Users\ADMIN>
```

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

def func():

x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied.

```
C: > Users > ADMIN > OneDrive >  
1  def func():  
2  x = 5  
3  y = 10  
4  return x+y
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop >  
1  def func():  
2      x = 5  
3      y = 10  
4      return x+y  
5  print(func())
```

OUTPUT:

```

PS C:\Users\ADMIN> & "C:/Program Files/Python
File "c:\Users\ADMIN\OneDrive\Desktop\ce
    x = 5
    ^
IndentationError: expected an indented block
PS C:\Users\ADMIN> & "C:/Program Files/Python
PS C:\Users\ADMIN> & "C:/Program Files/Python
15
PS C:\Users\ADMIN>

```

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

C: > Users > ADMIN > OneDrive > Desktop >

```

1  import maths
2  print(maths.sqrt(16))

```

Corrected code:

```

C: > Users > ADMIN > OneDrive > Desktop > certificates
1  import math
2  print(math.sqrt(16))
3

```

OUTPUT:

```

PS C:\Users\ADMIN> & "C:/Program Files/Python
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\cert
    import maths
ModuleNotFoundError: No module named 'maths'
PS C:\Users\ADMIN> & "C:/Program Files/Python
4.0
PS C:\Users\ADMIN>

```

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.

```

C: > Users > ADMIN > OneDrive > Desktop > certificates
1  def total(numbers):
2      for n in numbers:
3          return n
4  print(total([1,2,3]))
5

```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates >
1  def total(numbers):
2      total = 0
3      for n in numbers:
4          total += n
5      return total
6  print(total([1,2,3]))
7
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.
File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB
    for n in numbers:
    ^^^
IndentationError: expected an indented block after funct
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.
6
PS C:\Users\ADMIN> 
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

```
C: > Users > ADMIN > OneDrive > Desktop >
1  def calculate_area():
2      return length * width
3      print(calculate_area())
4
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.
1  def calculate_area(length, width):
2      return length * width
3      print(calculate_area(5,3))
4
```

OUTPUT:

```
File "c:\Users\ADMIN\OneDrive\Desktop\certificate
return length * width
^^^^^^

IndentationError: expected an indented block after
PS C:\Users\ADMIN> & "C:/Program Files/Python313/py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\certificate
    print(calculate_area())
    ~~~~~^
  File "c:\Users\ADMIN\OneDrive\Desktop\certificate
    return length * width
    ~~~~~^

NameError: name 'length' is not defined
PS C:\Users\ADMIN> & "C:/Program Files/Python313/py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\certificate
    print(calculate_area())
    ~~~~~^
  File "c:\Users\ADMIN\OneDrive\Desktop\certificate
    print(calculate_area(length,width))
    ~~~~~^

TypeError: calculate_area() missing 2 required posi
PS C:\Users\ADMIN> & "C:/Program Files/Python313/py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\certificate
    print(calculate_area(length,width))
    ~~~~~^

NameError: name 'length' is not defined
PS C:\Users\ADMIN> & "C:/Program Files/Python313/py
15
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():
    return 5 + "10"
print(add_values())
```

Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.

```
C: > Users > ADMIN > OneDrive > Desktop > certificates
1 def add_values():
2     return 5 + "10"
3     print(add_values())
4
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates
1 def add_values():
2     return 5 + 10
3     print(add_values())
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB 7.5.py", line 2
    return 5 + "10"
    ^^^^^
IndentationError: expected an indented block after function definition on line 1
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" "c:/Users/ADMIN/OneDrive/Desktop/certificates/LAB 7.5.py"
15
```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```
def combine():
    return "Numbers: " + [1, 2, 3]
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
C: > Users > ADMIN > OneDrive > Desktop > certificates
1 def combine():
2     return "Numbers: " + [1, 2, 3]
3     print(combine())
4
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.py > ...
1 def combine():
2     numbers = [1, 2, 3]
3     return "Numbers: " + str(numbers)
4 print(combine())
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/pytho
File "c:\Users\ADMIN\OneDrive\Desktop\certificates\L
return "Numbers: " + [1, 2, 3]
^^^^^^
IndentationError: expected an indented block after fun
PS C:\Users\ADMIN> & "C:/Program Files/Python313/pytho
PS C:\Users\ADMIN> & "C:/Program Files/Python313/pytho
Numbers: [1, 2, 3]
PS C:\Users\ADMIN> 
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```
def repeat_text():
```

```
    return "Hello" * 2.5
```

```
print(repeat_text())
```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.

```
C: > Users > ADMIN > OneDrive > Desktop
```

```
1 def repeat_text():
2     return "Hello" * 2.5
3     print(repeat_text())
4 
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificat
1 def repeat_text():
2     return "Hello" * 2
3     print(repeat_text())
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB 7\lab7.py", line 10, in <module>
    print(repeat_text())
          ~~~~~^
  File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB 7\lab7.py", line 15, in repeat_text
    return "Hello" * 2.5
          ~~~~~^~~~~
TypeError: can't multiply sequence by non-int of type 'float'
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
HelloHello
PS C:\Users\ADMIN>
```

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

```
def compute():
```

```
    value = None
```

```
    return value + 10
```

```
print(compute())
```

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.

```
C: > Users > ADMIN > OneDrive > Desktop > certificates
1  def compute():
2  value = None
3  return value + 10
4  print(compute())
5
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates
1  def compute():
2  value = 5
3  return value + 10
4  print(compute())
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
File "c:\Users\ADMIN\OneDrive\Desktop\certificates\LAB 7\lab7.py", line 2, in <module>
    value = None
          ^^^^^
IndentationError: expected an indented block after
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
15
PS C:\Users\ADMIN>
```

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():
a = input("Enter first number: ")
b = input("Enter second number: ")
return a + b
print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.

```
C: > Users > ADMIN > OneDrive > Desktop > certificates >
1  def sum_two_numbers():
2  a = input("Enter first number: ")
3  b = input("Enter second number: ")
4  return a + b
5  print(sum_two_numbers())
6
```

Corrected code:

```
C: > Users > ADMIN > OneDrive > Desktop > certificates > LAB 7.5.py >
1  def sum_two_numbers():
2      a = input("Enter first number: ")
3      b = input("Enter second number: ")
4      return int(a) + int(b)
5  print(sum_two_numbers())
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/
File "c:\Users\ADMIN\OneDrive\Desktop\certifica
a = input("Enter first number: ")
^
IndentationError: expected an indented block afte
PS C:\Users\ADMIN> & "C:/Program Files/Python313/
Enter first number: 5
Enter second number: 3
8
```